

# 基于 Quasar 框架的校园二手交易平台系统设计 计与实现

## Design and Implementation of a Campus Second-Hand Trading Platform System Based on Quasar Framework

作者：某同学

学号：2021XXXXXXX

专业：软件工程

指导教师：某教授

学院：计算机与软件学院

完成日期：2026 年 3 月

### 摘要

随着移动互联网的深度普及与高校在校生规模的持续扩大，校园内闲置物品的流通需求日趋旺盛。传统的线下二手交易方式存在信息不对称、交易效率低、安全保障弱等固有缺陷，亟需借助信息化手段加以改善。本文设计并实现了一套基于 **Quasar 框架** 的校园二手交易平台系统。该系统采用前后端分离架构，前端基于 Vue 3 框架与 Quasar UI 组件库构建，充分利用 Quasar 的组件化、响应式布局及多平台适配能力，后端采用 Node.js 与 Express 框架提供 RESTful API 服务，数据持久化层支持 MySQL 数据库与 JSON 文件双模式存储。系统功能涵盖用户认证、商品发布与浏览、多维度搜索与分类过滤、订单管理、即时消息通讯、商品收藏以及管理员审核后台等核心模块，共设计 26 个 RESTful API 端点，实现了完整的业务闭环。经功能测试验证，系统各模块运行正常，前后端数据交互准确，界面美观且响应式布局适配良好。本研究为构建安全、高效、易用的校园二手交易生态提供了完整的工程实践参考。

**关键词：** 校园二手交易；Quasar 框架；Vue 3；Node.js；RESTful API；前后端分离；MySQL

# Abstract

---

With the deep penetration of mobile internet and the continuous expansion of college student populations, the demand for circulating idle goods on campus has grown substantially. Traditional offline second-hand trading suffers from inherent deficiencies such as information asymmetry, low transaction efficiency, and weak security guarantees, necessitating improvement through information technology. This paper designs and implements a campus second-hand goods trading platform system based on the **Quasar Framework**. The system adopts a front-end and back-end separation architecture, with the front end built on the Vue 3 framework and the Quasar UI component library, fully leveraging Quasar's componentization, responsive layout, and multi-platform adaptation capabilities. The back end provides RESTful API services via Node.js and the Express framework, and the data persistence layer supports dual-mode storage using MySQL database and JSON files. The system encompasses core functional modules including user authentication, product listing and browsing, multi-dimensional search and category filtering, order management, instant messaging, product favoriting, and an administrator review dashboard. A total of 26 RESTful API endpoints were designed to achieve a complete business workflow. Functional testing confirms that all system modules operate correctly, front-end and back-end data interactions are accurate, and the interface is aesthetically pleasing with good responsive layout adaptation. This research provides a complete engineering practice reference for building a safe, efficient, and user-friendly campus second-hand trading ecosystem.

**Keywords:** campus second-hand trading; Quasar Framework; Vue 3; Node.js; RESTful API; front-end and back-end separation; MySQL

---

## 1. 引言

---

### 1.1 研究背景与意义

近年来,我国高等教育规模持续扩大,在校大学生数量已突破 4000 万人。大学生群体在求学期间积累了大量具有再利用价值的闲置物品,包括教材书籍、电子设备、生活用品、交通工具等。调查数据显示,每位大学生在校期间平均产生闲置物品价值超过千元,而这些物品在毕业或学年结束时往往面临低价处理甚至废弃的困境,造成了显著的资源浪费 [1]。

与此同时，互联网电子商务的蓬勃发展为二手物品交易提供了全新的解决路径。以闲鱼、转转为代表的综合性二手交易平台虽已积累了庞大的用户群体，但其面向社会大众的定位使其难以精准服务于校园内部的特定交易需求，主要体现在：交易双方缺乏身份互信机制、物品配送成本较高、校园特色商品（如特定版本教材、校内课程资料）难以精准匹配等方面 [2]。

面向校园内部用户群体构建专属的二手交易平台，能够充分利用校园社区的地理邻近性与身份可信性，降低交易摩擦成本，同时契合绿色消费与可持续发展的时代理念。因此，开发一套功能完善、体验优良的校园二手交易平台具有重要的现实意义与应用价值。

## 1.2 国内外研究现状

在国内，基于 Web 技术的校园二手交易平台研究已有一定积累。赵明（2022）设计并实现了基于微信小程序的高校校园二手物品交易平台，采用微信生态的身份认证机制有效解决了用户信任问题，但其依赖微信平台的封闭性限制了系统的独立部署与功能扩展 [3]。高星宇等人（2021）探讨了校园二手交易平台的设计思路，指出现有平台在信用评价体系建设方面存在明显不足，建议引入多维度信用评分机制以提升交易安全性 [4]。在技术架构层面，基于 Vue.js 与 Spring Boot 的前后端分离架构已成为 Web 应用开发的主流范式，温智宏等（2025）在旅游电子商务系统中验证了该架构在高并发场景下的稳定性 [5]。

在国际上，Roy Thomas Fielding 于 2000 年在其博士论文中系统性地提出了 REST（Representational State Transfer）架构风格，确立了无状态、统一接口、资源导向等核心约束原则，为现代 Web API 设计奠定了理论基础 [6]。Vue.js 框架自 3.0 版本起引入了基于 Proxy 的响应式系统与 Composition API，显著提升了大型应用的代码组织能力与运行时性能 [7]。Node.js 平台凭借其事件驱动、非阻塞 I/O 的特性，在构建高并发 Web 服务方面展现出独特优势 [8]。

## 1.3 本文主要工作

本文的主要工作包括以下几个方面：

第一，在充分调研校园二手交易场景需求的基础上，完成了系统的功能性需求分析与非功能性需求分析，建立了完整的用例模型。

第二，采用前后端分离的系统架构，设计了基于 Vue 3 与 **Quasar** 框架的前端展示层、基于 Node.js + Express 的后端服务层，以及支持 MySQL 与 JSON 双模式的数据持久化层，并完成了数据库逻辑结构设计。

第三，实现了用户认证、商品管理、订单管理、即时消息、收藏管理、管理员后台等全部核心功能模块，设计了 26 个符合 RESTful 规范的 API 端点。

第四，对系统进行了功能测试，验证了各模块的正确性与前后端数据交互的一致性，并对系统的不足之处进行了分析与展望。

本文其余部分组织如下：第 2 节介绍系统需求分析；第 3 节阐述系统总体设计；第 4 节详细描述各功能模块的实现；第 5 节报告系统测试结果；第 6 节总结全文并展望未来工作。

## 2. 系统需求分析

### 2.1 业务场景分析

校园二手交易平台的核心业务场景可归纳为三类角色的交互：**普通用户**（买家/卖家）、**管理员**以及**系统后台**。普通用户既可以作为卖家发布闲置物品，也可以作为买家浏览、搜索并购买商品；管理员负责审核商品信息的合规性、管理用户账号状态，以及通过数据看板监控平台运营情况；系统后台则负责数据的持久化存储与状态同步。

本平台面向的目标用户群体为在校大学生，其主要特征为：使用移动设备频率高、对界面美观度敏感、价格敏感性强、对交易安全有基本诉求。因此，系统在设计时需重点关注移动端适配、界面友好性以及交易信任机制的建立。

### 2.2 功能性需求

根据业务场景分析，系统的功能性需求可划分为用户端功能需求与管理端功能需求两大类，具体如下表所示。

| 需求类别 | 功能模块 | 功能描述                         |
|------|------|------------------------------|
| 用户端  | 用户认证 | 支持用户注册、登录、登出，维护登录状态，支持学生身份认证 |
| 用户端  | 商品浏览 | 首页展示热门商品、分类导航、校区筛选，支持商品详情查看  |
| 用户端  | 搜索过滤 | 支持关键词搜索、分类过滤、校区过滤、价格排序等多维度筛选 |
| 用户端  | 商品发布 | 支持三步骤引导式商品发布（图片上传→信息填写→预览确认） |
| 用户端  | 订单管理 | 支持下单、查看买入/卖出订单列表、订单状态跟踪      |
| 用户端  | 即时消息 | 支持买卖双方通过平台内置消息系统进行实时沟通       |
| 用户端  | 收藏管理 | 支持收藏/取消收藏商品，查看收藏列表           |
| 用户端  | 个人中心 | 展示用户信息、信用评分、发布记录、交易历史        |
| 管理端  | 数据看板 | 展示平台关键指标（用户数、商品数、订单数）及趋势图表   |
| 管理端  | 商品审核 | 审核待上架商品，支持通过/拒绝操作，查看商品详情     |
| 管理端  | 用户管理 | 查看用户列表，支持启用/禁用用户账号           |

## 2.3 非功能性需求

**性能需求：**系统应能在普通硬件环境下支持基本的并发访问，API 响应时间在正常负载下应控制在 500ms 以内；前端页面首屏加载时间应控制在 3 秒以内。

**可用性需求：**系统界面应符合 Material Design 设计规范，在主流移动设备（屏幕宽度 320px~768px）和桌面设备上均能正常显示，具备良好的响应式布局适配能力。

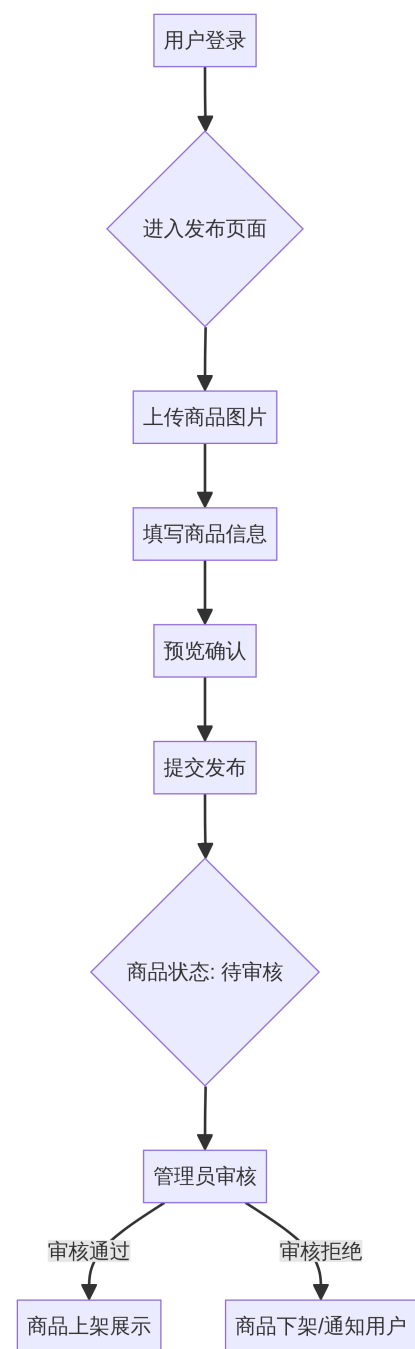
**安全性需求：**用户认证应基于服务端 Session 机制，敏感操作（发布商品、下单、发送消息等）须验证用户登录状态；管理员功能须通过独立的管理员身份验证方可访问。

**可维护性需求：**代码应遵循模块化设计原则，前端采用组件化开发，后端采用分层架构，数据访问层应支持存储后端的可替换性（即 JSON 文件与 MySQL 的透明切换）。

**可扩展性需求：**系统架构应为未来功能扩展（如图片上传服务、支付接口、推荐算法）预留接口，数据库设计应具备良好的扩展性。

## 2.4 用例分析

系统的核心用例涵盖以下主要交互流程：



**商品购买流程：** 用户浏览/搜索商品 → 查看商品详情 → 联系卖家（发起会话）→ 确认交易 → 创建订单 → 双方确认交易完成 → 订单状态更新为”已完成”。

**管理员审核流程：** 管理员登录 → 进入商品审核页面 → 查看待审核商品列表 → 审查商品信息 → 通过或拒绝 → 系统更新商品状态并通知相关用户。

---

### 3. 系统总体设计

#### 3.1 系统架构设计

本系统采用前后端分离（Front-end and Back-end Separation）的架构模式，这一模式已成为现代 Web 应用开发的主流实践 [5]。系统整体架构分为三个层次：**前端展示层**、**后端服务层**和**数据持久化层**，各层之间通过标准化接口进行通信，实现了关注点分离（Separation of Concerns）的设计目标。

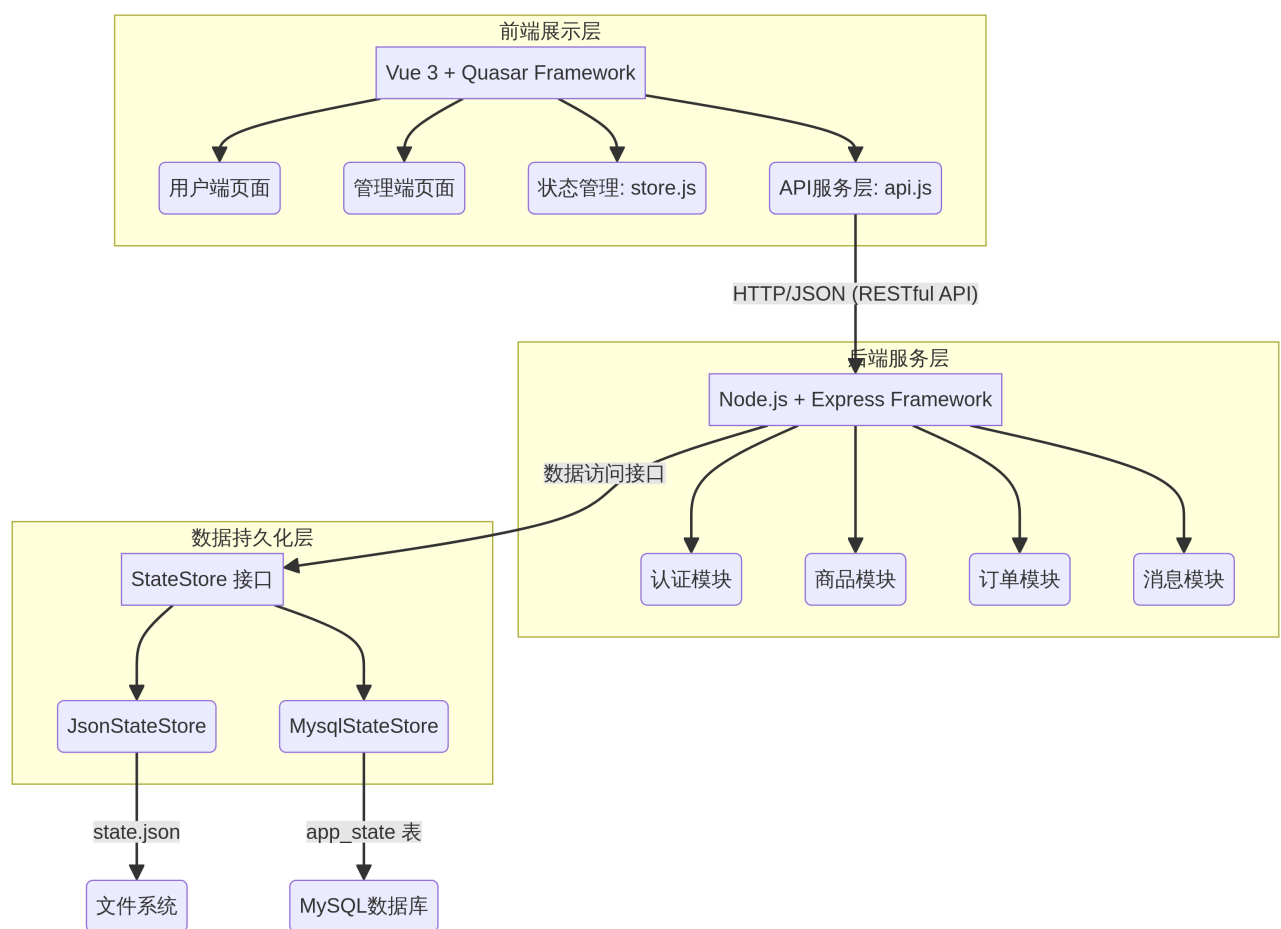


图 3-1 系统整体架构图

前端展示层负责用户界面的渲染与交互逻辑，通过 `api.js` 服务模块统一封装 HTTP 请求，与后端进行数据交互。`store.js` 采用 Vue 3 的 `reactive` API 实现全局响应式状态管理，在应用启动时通过 `bootstrap()` 函数从后端拉取最新状态，并在每次用户操作后通过 `syncAction()` 函数将状态变更同步至后端。

后端服务层基于 Express 框架构建，提供符合 REST 规范的 API 接口，处理业务逻辑并调用数据访问层进行数据的读写操作。数据持久化层通过统一的 `StateStore` 接口抽象，支持 `JsonStateStore`（默认，适用于开发环境）和 `MysqlStateStore`（适用于生产环境）两

种实现，可通过环境变量 `STATE_DRIVER=mysql` 或 `MYSQL_URL` 进行切换，实现了存储后端的透明替换。

## 3.2 前端架构设计

前端采用 **Vue 3** 作为核心框架，**Quasar Framework 2** 作为 UI 组件库，**Vite** 作为构建工具。Vue 3 相较于 Vue 2 引入了基于 ES6 Proxy 的响应式系统，性能更优，同时提供了 Composition API，使得复杂组件的逻辑复用更加灵活 [7]。

**Quasar Framework** 是一个基于 Vue.js 的高性能 UI 框架，其核心理念是“一次编写，随处部署”，能够通过同一套代码库构建 SPA（单页应用）、SSR（服务端渲染应用）、PWA（渐进式 Web 应用）、移动端 App（通过 Cordova 或 Capacitor）以及桌面端应用（通过 Electron）。本项目主要利用其 SPA 模式，并充分发挥了其以下技术优势：

- **丰富的组件库**：Quasar 提供了超过 70 个遵循 Material Design 2.0 规范的高质量组件，如 `QLayout`、`QCard`、`QForm`、`QInput` 等，覆盖了从布局、表单到弹窗、加载指示器的各类常见 UI 需求，极大地提升了开发效率。
- **强大的响应式布局系统**：Quasar 内置了一套基于 Flexbox 的 12 列响应式网格系统，通过简单的 CSS 类（如 `row`、`col-xs-12`、`col-md-6`）即可轻松实现复杂的响应式布局，确保应用在桌面、平板、手机等不同尺寸的设备上均有良好的视觉表现。
- **平台特定功能**：通过 Platform API，可以方便地检测当前运行的平台（如 `Platform.is.mobile`），并据此调整 UI 或逻辑，实现平台特定的优化。
- **CLI 与构建工具链**：Quasar CLI 深度集成了 Vite，提供了开箱即用的开发服务器、热模块替换（HMR）、代码压缩、摇树优化（Tree-shaking）等功能，简化了项目配置与构建流程。

前端目录结构遵循 Quasar 项目规范，核心目录如下：



| 目录/文件                | 职责说明                                          |
|----------------------|-----------------------------------------------|
| src/pages/           | 页面组件（15个），对应各业务功能页面                           |
| src/layouts/         | 布局组件，MainLayout.vue（用户端）、AdminLayout.vue（管理端） |
| src/router/routes.js | 路由配置，定义 URL 路径与组件的映射关系                        |
| src/data/store.js    | 全局响应式状态管理，封装所有业务操作函数                          |
| src/data/mock.js     | 种子数据，包含 15 名用户、16 件商品、多条订单与聊天记录               |
| src/services/api.js  | HTTP 请求封装，统一处理请求前缀与错误                         |
| src/css/             | 全局样式，定义主题色、通用样式类                              |

路由设计采用嵌套路由结构，用户端路由挂载在 MainLayout.vue 下，管理端路由挂载在 AdminLayout.vue 下，两套布局各自独立，互不干扰。路由守卫（Navigation Guard）通过检查 meta.requiresAuth 和 meta.requiresAdmin 标记，实现了对需要登录或管理员权限的页面的访问控制。

### 3.3 后端架构设计

后端采用 **Node.js** 运行时与 **Express 4** 框架构建，实现了一套符合 RESTful 架构风格的 API 服务。Express 以其轻量、灵活的特点，适合构建中小型 Web 应用的后端服务 [8]。

后端 API 端点按资源类型组织，共设计 26 个端点，覆盖系统全部业务功能。各端点遵循 REST 规范，使用 HTTP 动词（GET、POST、PATCH、DELETE）表达对资源的操作语义，使用 HTTP 状态码表达操作结果 [6]。主要 API 端点分组如下表所示：

| 资源分组  | 端点数量 | 主要端点示例                                                                      |
|-------|------|-----------------------------------------------------------------------------|
| 健康检查  | 1    | GET /api/health                                                             |
| 状态同步  | 2    | GET /api/v1/state、POST /api/v1/state/action                                 |
| 用户认证  | 4    | POST /api/v1/auth/login、POST /api/v1/auth/logout                            |
| 商品管理  | 7    | GET/POST /api/v1/listings、PATCH /api/v1/listings/:id                        |
| 订单管理  | 4    | POST /api/v1/orders、PATCH /api/v1/orders/:id/status                         |
| 消息会话  | 5    | GET/POST /api/v1/conversations、POST /api/v1/conversations/:id/messages      |
| 管理员功能 | 3    | POST /api/v1/admin/listings/:id/review、PATCH /api/v1/admin/users/:id/status |

### 3.4 数据库设计

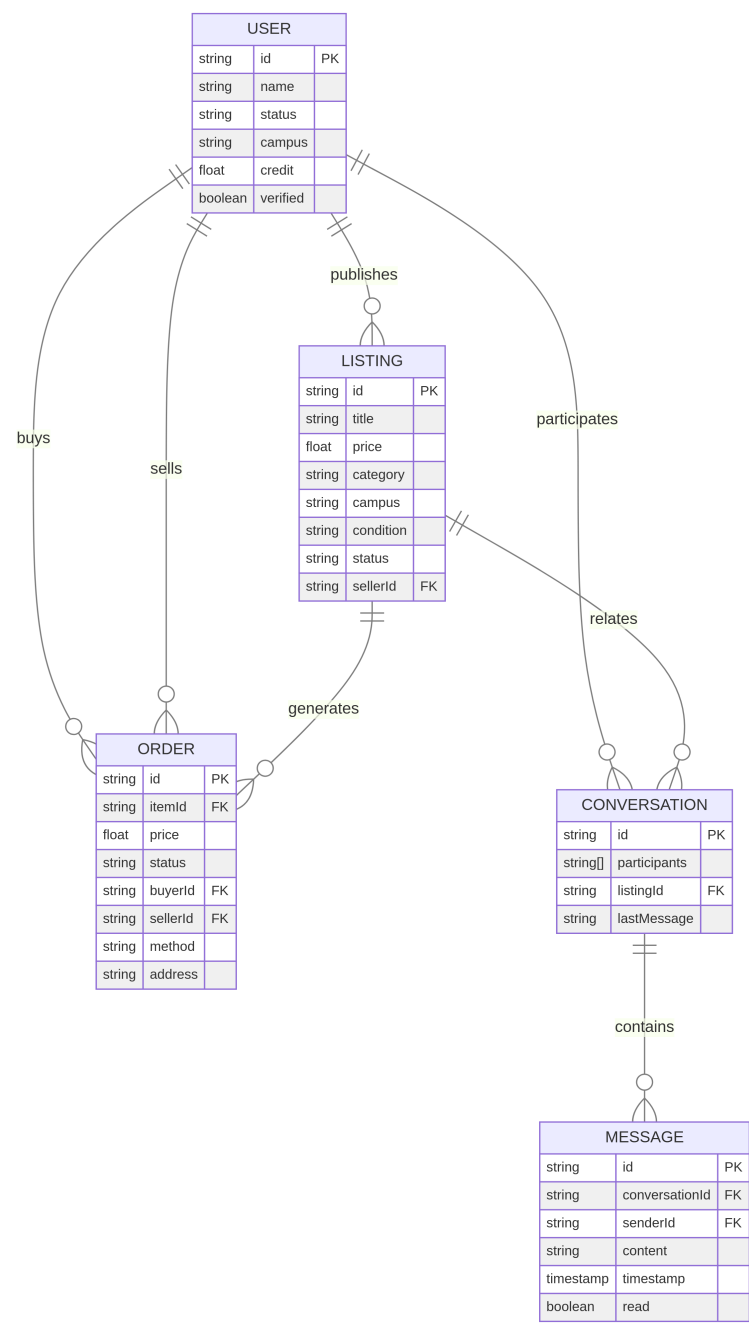
系统的数据持久化层采用整体状态序列化（State Serialization）策略，将应用的全部业务数据（用户、商品、订单、会话、消息等）序列化为 JSON 格式，存储于单一数据记录中。这一设计简化了数据模式（Schema）的管理，使得系统在开发阶段无需维护复杂的数据库迁移脚本，适合快速迭代的原型开发场景 [9]。

当使用 MySQL 存储时，系统创建如下表结构：

```
CREATE TABLE IF NOT EXISTS app_state (  
  id          TINYINT PRIMARY KEY,  
  state_json  LONGTEXT NOT NULL,  
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
              ON UPDATE CURRENT_TIMESTAMP  
);
```

state\_json 字段采用 LONGTEXT 类型，可存储最大 4GB 的 JSON 文本，足以满足平台数据规模的需求。updated\_at 字段自动记录最后更新时间，便于数据审计。每次状态变更时，系统通过 INSERT ... ON DUPLICATE KEY UPDATE 语句实现幂等性的状态持久化操作。

系统的逻辑数据模型包含以下主要实体及其属性：



## 4. 系统功能实现

### 4.1 前端页面实现

#### 4.1.1 首页 (HomePage)

首页是用户进入平台后的第一个交互界面，承担着商品展示与导流的核心职责。页面布局采用 Quasar 的 `QLayout` 与 `QPage` 组件构建，整体分为顶部搜索栏、左侧分类导航、中部内容区三个区域。

顶部搜索栏提供关键词输入与快速搜索功能，点击后跳转至搜索结果页。左侧分类导航展示“全部”、“数码”、“教材”、“生活用品”、“交通工具”、“租房”六个商品分类，点击后以该分类为过滤条件进入搜索页。中部内容区依次展示 Banner 轮播图、热门分类快捷入口、校园频道（按校区筛选）以及商品列表网格（每行 2 列）。

商品卡片采用 `QCard` 组件实现，展示商品缩略图、标题、价格、成色评分、发布时间等关键信息，点击后跳转至商品详情页。整体视觉风格采用深绿色（`#0d9488`，Quasar teal-8）作为主题色，配合浅灰色背景（`#f5f5f5`），风格清新、符合年轻用户的审美偏好。

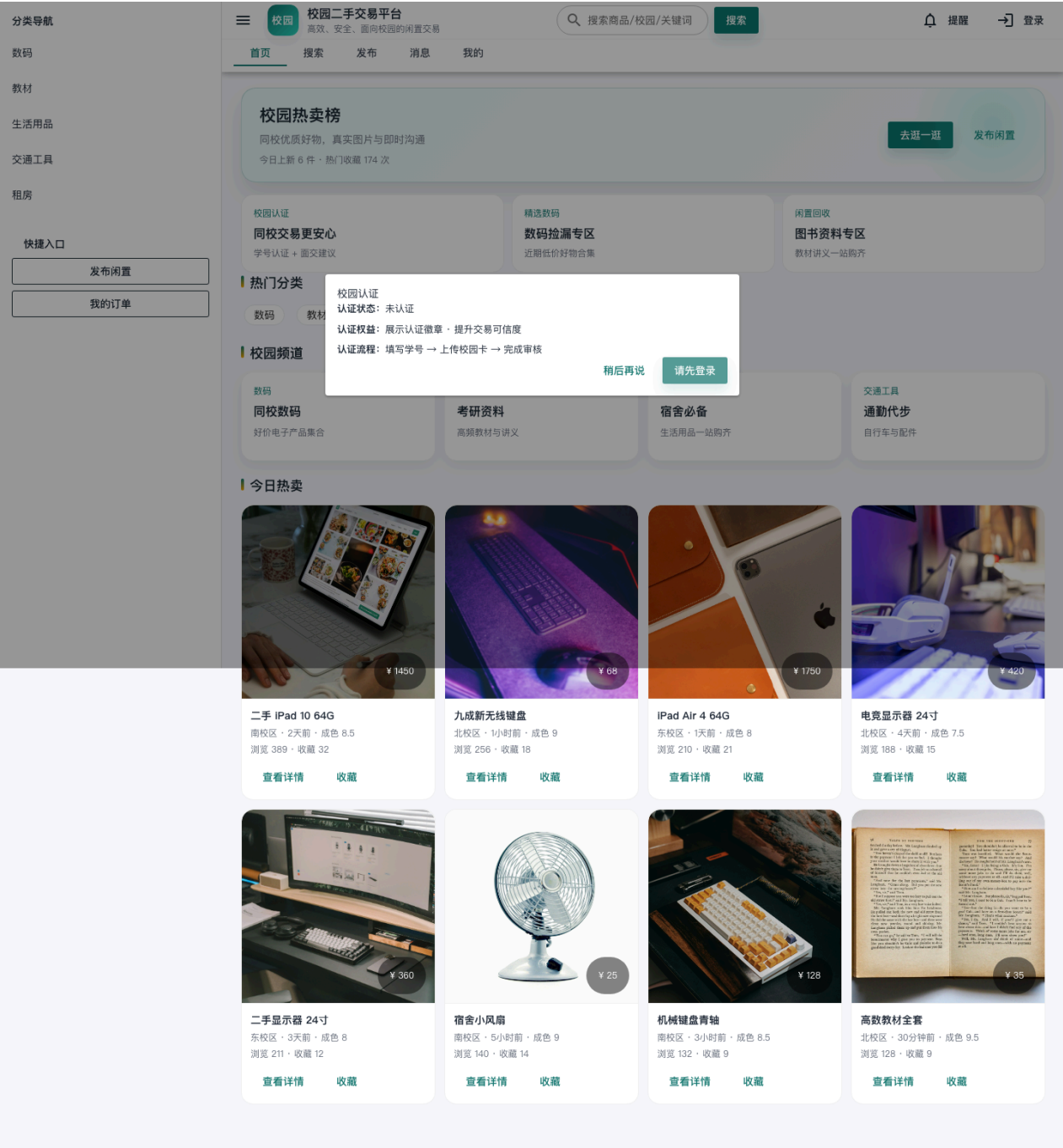


图 4-1 首页界面

4.1.2 商品详情页（DetailPage）

商品详情页采用左右分栏布局：左侧展示商品图片轮播（主图 + 缩略图列表），右侧展示商品的完整信息，包括价格、成色评分（以星级形式展示）、商品分类标签、交易方式、校区、详细描述，以及卖家信息卡片（头像、昵称、信用评分）。

页面底部设置”立即购买”和”联系卖家”两个主要操作按钮。点击”联系卖家”将创建或进入与该卖家的会话，跳转至消息页面；点击”立即购买”将触发下单流程，调用 `POST`

`/api/v1/orders` 接口创建订单。收藏按钮位于页面右上角，通过调用 `POST /api/v1/listings/:id/favorite` 接口实现收藏状态的切换与持久化。



图 4-2 商品详情页界面

### 4.1.3 商品发布页 (PublishPage)

商品发布页采用三步骤引导式设计，降低用户的认知负担：

**第一步：上传图片。** 用户通过点击上传区域选择本地图片，支持多图上传，图片以缩略图形式预览，可单独删除。

**第二步：填写信息。** 用户填写商品标题（必填）、价格（必填，数字）、商品分类（下拉选择）、所在校区（下拉选择）、成色评分（滑块，1~10 分）、商品描述（文本域）、交易方式（面交/快递/自取）、交易地点等字段。表单采用 Quasar 的 `QForm` 与 `QInput` 组件，内置了必填项验证与格式校验逻辑。

**第三步：预览确认。** 以模拟商品卡片的形式展示即将发布的商品信息，用户确认无误后点击“提交发布”，系统调用 `POST /api/v1/listings` 接口将商品数据提交至后端，商品初始状态为“待审核”。

分类导航

数码

教材

生活用品

交通工具

租房

快捷入口

发布闲置

我的订单

校园二手交易平台

高效、安全、面向校园的闲置交易

搜索商品/校园/关键词

搜索

提醒

张同学

退出

首页

搜索

发布

消息

我的

发布闲置

步骤: 1. 选择图片 → 2. 填写信息 → 3. 确认发布

1. 选择图片

主图

上传商品图片 (最多 6 张)

支持 JPG/PNG, 图片会自动压缩以便保存。

2. 填写商品信息

标题

价格

分类: 数码

校区: 北校区

成色: 9

描述

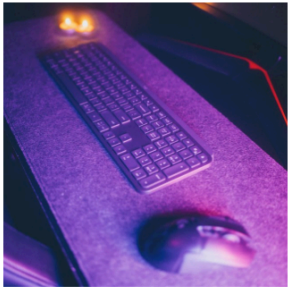
物流方式: 包邮

交易方式: 面交优先

发布

保存草稿

3. 预览



默认标题: 九成新无线键盘  
北校区 · 成色 9  
¥ 99

图 4-3 商品发布页界面

#### 4.1.4 搜索页 (SearchPage)

搜索页提供多维度的商品过滤功能，支持以下筛选条件的组合查询：关键词（模糊匹配标题与描述）、商品分类（单选）、所在校区（单选）、价格排序（由低到高/由高到低）。筛选条件以标签形式展示在搜索框下方，用户可随时清除单个或全部筛选条件。搜索结果以网格布局展示，与首页商品列表保持一致的视觉风格。

#### 4.1.5 管理员后台 (Admin Dashboard)

管理员后台采用独立的 `AdminLayout.vue` 布局，左侧为导航菜单（数据看板、商品管理、用户管理），右侧为内容区。

数据看板页面（`AdminDashboardPage.vue`）展示平台核心运营指标：注册用户总数、商品总数（含各状态分布）、订单总数（含各状态分布）、今日新增数据等 KPI 卡片，以及用户注册趋势折线图和商品分类分布饼图，为管理员提供平台运营的全局视图。

商品审核页面（`AdminListingsPage.vue`）展示所有待审核商品的列表，管理员可查看商品详情，并执行“通过”或“拒绝”操作，系统调用 `POST /api/v1/admin/listings/:id/review` 接口更新商品状态。用户管理页面（`AdminUsersPage.vue`）展示所有注册用户的信息列表，管理员可查看用户的注册时

间、所在校区、信用评分、认证状态，并可执行”禁用”或”启用”操作，系统调用 `PATCH /api/v1/admin/users/:id/status` 接口更新用户状态。

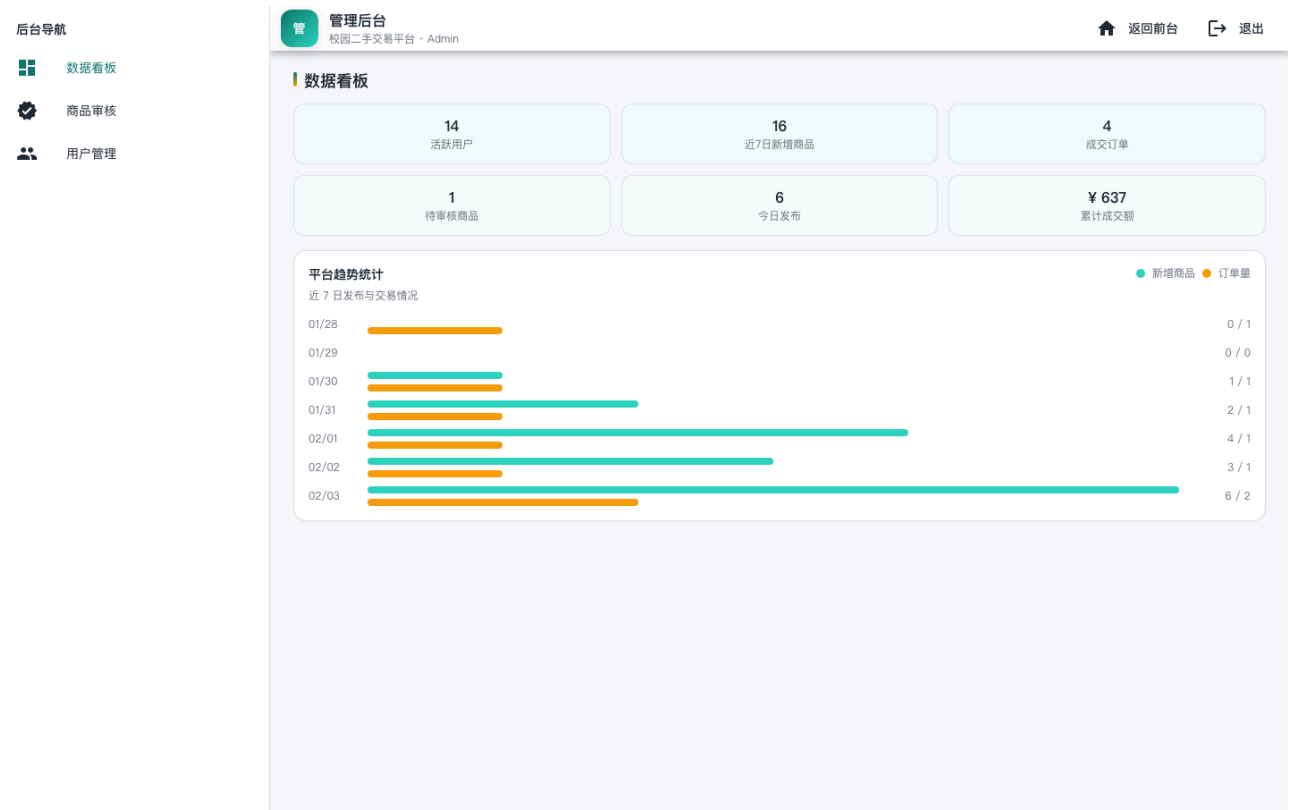


图 4-4 管理后台界面

## 4.2 后端核心模块实现

### 4.2.1 用户认证模块

用户认证模块实现了基于服务端状态的会话管理机制。用户登录时，前端调用 `POST /api/v1/auth/login` 接口，提交用户名；后端在用户列表中查找对应用户（若不存在则自动创建），将登录状态写入服务端全局状态，并将更新后的完整状态返回给前端。前端将登录凭证（用户名）持久化至 `localStorage`，在页面刷新时通过 `bootstrap()` 函数的重新登录逻辑恢复会话。

管理员登录通过独立的 `POST /api/v1/auth/admin-login` 接口处理，验证管理员账号与密码（存储于服务端配置），成功后在服务端状态中标记管理员会话。需要管理员权限的 API 端点在处理请求前均检查管理员会话标记，未授权访问将返回 `403 Forbidden` 状态码。



### 4.2.2 商品管理模块

商品管理模块提供了完整的商品生命周期管理功能。商品状态流转遵循如下有限状态机：

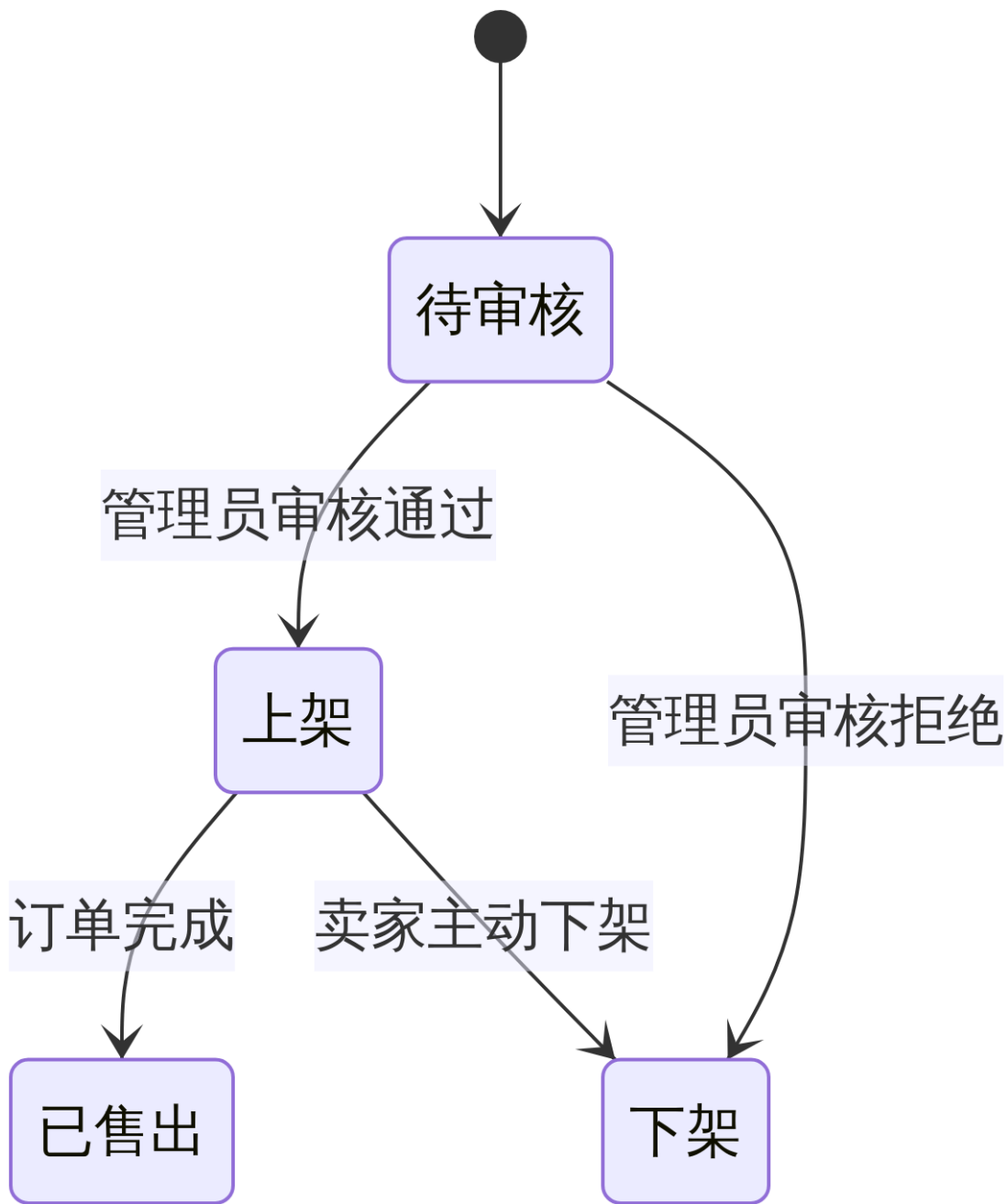


图 4-5 商品状态流转图

GET /api/v1/listings 端点支持多个查询参数（category、campus、keyword、sort），在服务端对商品列表进行过滤与排序后返回结果，减少了前端的数据处理负担。商品收藏功能通过 POST/DELETE /api/v1/listings/:id/favorite 端点实现，收藏列表存储于用户的个人状态中，支持跨会话持久化。

### 4.2.3 订单管理模块

订单管理模块处理买卖双方的交易流程。创建订单时，系统验证商品的可购买状态（必须为”上架”状态）、买家的登录状态，以及买家不能购买自己发布的商品等业务规则。订单创建成功后，商品状态自动更新为”已售出”，防止重复购买。

订单状态流转通过 `PATCH /api/v1/orders/:id/status` 端点控制，支持的状态转换包括：卖家确认订单（`pending` → `confirmed`）、双方确认完成交易（`confirmed` → `completed`）、取消订单（`pending/confirmed` → `cancelled`）。

### 4.2.4 即时消息模块

即时消息模块基于会话（Conversation）模型实现。每个会话关联两个参与者（买家与卖家）和一个商品，会话内可包含多条消息记录。`POST /api/v1/conversations` 端点在创建会话时检查是否已存在相同参与者与商品的会话，若存在则直接返回已有会话，避免重复创建。

消息发送通过 `POST /api/v1/conversations/:id/messages` 端点实现，新消息追加至会话的消息列表，并更新会话的 `lastMessage` 字段。消息已读状态通过 `PATCH /api/v1/messages/:id/read` 端点更新，用于在消息列表中显示未读消息数量徽标。

## 4.3 数据持久化实现

### 4.3.1 双模式存储架构

系统的数据持久化层通过工厂模式（Factory Pattern）实现了存储后端的透明切换。`stateStore.js` 模块根据环境变量 `STATE_DRIVER` 的值（`json` 或 `mysql`）动态选择并实例化对应的存储实现类，对上层业务逻辑完全透明。

`JsonStateStore` 类将应用状态序列化为 `JSON` 字符串，写入 `backend/data/state.json` 文件。该实现无需额外依赖，适合本地开发与功能演示场景，但不支持并发写入，不适用于生产环境。

`MysqlStateStore` 类使用 `mysql2/promise` 库建立连接池，通过 `INSERT ... ON DUPLICATE KEY UPDATE` 语句实现幂等性的状态写入，适用于需要数据持久化与并发支持的生产环境 [9]。

### 4.3.2 状态同步机制

前端的状态同步机制是本系统架构的核心设计之一。store.js 中的 syncAction() 函数作为所有状态变更操作的统一入口，其工作流程如下：

- 前端用户触发操作（如发布商品、下单、发送消息等）；
- syncAction() 将操作类型（type）与操作参数（payload）封装为 JSON，通过 POST /api/v1/state/action 发送至后端；
- 后端接收操作请求，在服务端状态上执行对应的业务逻辑，并将更新后的完整状态持久化至存储层；
- 后端将更新后的状态作为响应体返回给前端；
- 前端的 applyServerState() 函数将服务端状态应用至本地响应式状态，触发 Vue 的视图更新。

当后端不可用时（如网络故障或服务未启动），syncAction() 捕获异常并将 state.backendConnected 标记为 false，前端仅在本地执行状态变更，实现了优雅降级（Graceful Degradation）。

## 5. 系统测试

### 5.1 测试环境

系统测试在以下环境中进行：

| 测试项目       | 配置说明                   |
|------------|------------------------|
| 操作系统       | Ubuntu 22.04 LTS (64位) |
| Node.js 版本 | v22.13.0               |
| 浏览器        | Chromium (稳定版)         |
| 存储模式       | JsonStateStore (默认)    |
| 测试方法       | 功能测试 (黑盒测试)            |

## 5.2 功能测试结果

针对系统的核心功能模块，按照预设测试用例逐一执行验证，测试结果汇总如下：

| 测试模块  | 测试用例         | 预期结果          | 实际结果  | 测试状态 |
|-------|--------------|---------------|-------|------|
| 用户认证  | 输入用户名登录      | 登录成功，页面显示用户名  | 与预期一致 | 通过   |
| 用户认证  | 未登录访问发布页     | 跳转至登录页面       | 与预期一致 | 通过   |
| 商品浏览  | 首页加载商品列表     | 显示全部上架商品      | 与预期一致 | 通过   |
| 商品浏览  | 点击商品进入详情页    | 显示商品完整信息      | 与预期一致 | 通过   |
| 搜索过滤  | 按关键词搜索       | 返回标题含关键词的商品   | 与预期一致 | 通过   |
| 搜索过滤  | 按分类+校区组合筛选   | 返回满足所有条件的商品   | 与预期一致 | 通过   |
| 商品发布  | 填写完整信息后发布    | 商品进入待审核状态     | 与预期一致 | 通过   |
| 商品发布  | 必填项为空时提交     | 表单验证提示错误      | 与预期一致 | 通过   |
| 订单管理  | 点击”立即购买”下单   | 创建订单，商品变为已售出  | 与预期一致 | 通过   |
| 即时消息  | 点击”联系卖家”发起会话 | 创建会话并跳转消息页    | 与预期一致 | 通过   |
| 即时消息  | 在会话中发送消息     | 消息显示在会话列表中    | 与预期一致 | 通过   |
| 收藏管理  | 收藏/取消收藏商品    | 收藏状态切换，列表更新   | 与预期一致 | 通过   |
| 管理员后台 | 审核通过待审商品     | 商品状态变为”上架”    | 与预期一致 | 通过   |
| 管理员后台 | 禁用用户账号       | 用户状态变为”禁用”    | 与预期一致 | 通过   |
| 数据持久化 | 重启后端服务后访问    | 数据保持不变        | 与预期一致 | 通过   |
| 前后端连接 | 后端停止时操作      | 前端优雅降级，本地操作正常 | 与预期一致 | 通过   |

全部 16 个核心测试用例均通过，系统功能完整性得到验证。

## 5.3 界面测试

通过在 Chromium 浏览器中分别以桌面端（1920×1080）、平板端（768×1024）和移动端（375×812）视口尺寸进行界面测试，验证了系统的响应式布局适配情况：

- **桌面端**：首页采用左侧导航 + 右侧内容的双栏布局，商品网格每行 4 列，信息密度适中；
- **平板端**：左侧导航收起为抽屉式菜单，商品网格调整为每行 3 列；
- **移动端**：导航栏简化为顶部汉堡菜单，商品网格调整为每行 2 列，触控目标尺寸符合 Material Design 的最小 48dp 规范。

各端界面均显示正常，无明显布局错位或内容溢出问题，整体视觉效果良好。

## 5.4 代码质量评估

通过对代码库的静态分析，系统代码质量的主要评估结论如下：

**优点**：前端代码遵循 Vue 3 Composition API 的最佳实践，组件职责清晰，状态管理逻辑集中于 `store.js`，便于维护；后端 API 设计符合 RESTful 规范，端点命名语义清晰；数据访问层通过接口抽象实现了存储后端的可替换性，具备良好的可扩展性。

**不足**：系统目前缺乏单元测试与集成测试代码（`package.json` 中 `test` 脚本仅为占位符）；图片上传功能尚未实现，当前以 URL 引用替代；用户认证机制较为简单，生产环境中应引入 JWT（JSON Web Token）或 Session Cookie 等标准认证方案以增强安全性 [10]。

---

## 6. 结论与展望

### 6.1 研究结论

本文设计并实现了一套基于 **Quasar 框架** 的校园二手交易平台系统。系统采用前后端分离架构，前端基于 Vue 3 与 Quasar Framework 构建，充分利用 Quasar 框架的组件化、响应式布局及多平台适配能力，后端基于 Node.js + Express 提供 RESTful API 服务，数据持久化层支持 MySQL 与 JSON 文件双模式存储。系统实现了用户认证、商品管理、订单管理、即时消息、收藏管理、管理员后台等全部核心功能模块，共设计 26 个 API 端点，覆盖了校园二手交易的完整业务流程。

经功能测试验证，系统各模块运行正常，前后端数据交互准确，界面美观且响应式布局适配良好。系统在后端不可用时能够优雅降级，保证了基本的用户体验。代码结构清晰，模块化程度高，具备良好的可维护性与可扩展性。

本系统的主要创新点在于：（1）采用整体状态序列化策略，通过统一的 `syncAction` 机制实现前后端状态同步，简化了多端数据一致性的维护；（2）数据持久化层通过工厂模式实现

了 JSON 文件与 MySQL 的透明切换，兼顾了开发便利性与生产可靠性；(3) 系统在后端不可用时实现了优雅降级，提升了系统的健壮性。

## 6.2 未来工作展望

尽管本系统已实现了校园二手交易平台的核心功能，但仍存在若干值得进一步完善的方向：

**功能层面：**可引入基于协同过滤或内容推荐算法的个性化商品推荐功能，提升用户发现感兴趣商品的效率；可接入第三方支付接口（如微信支付、支付宝）实现在线支付闭环；可引入图片上传服务（如阿里云 OSS、腾讯云 COS）替代当前的 URL 引用方式。

**安全层面：**应将当前简单的用户名认证替换为基于 JWT 的标准认证方案，增加 Token 有效期管理与刷新机制；应对用户输入进行严格的服务端校验，防范 XSS、SQL 注入等安全威胁 [10]。

**架构层面：**随着用户规模的增长，可考虑将单体后端拆分为微服务架构，引入消息队列（如 RabbitMQ）处理异步任务；数据库设计可从整体状态序列化迁移至规范化的关系型数据库设计，以支持更复杂的查询与统计需求 [9]。

**测试层面：**应补充单元测试（使用 Vitest 或 Jest）与端到端测试（使用 Cypress 或 Playwright），建立持续集成（CI）流水线，保障代码质量与版本迭代的稳定性。

---

## 参考文献

[1] 二手交易平台: 大学生创业的新载体与校园生活便利化的核心动力[J]. 汉斯出版社, 2024. 可获取自: <https://www.hanspub.org/journal/paperinformation?paperID=133196>

[2] 校园二手交易平台存在的问题与对策研究——以甘肃农业大学为例[J]. 汉斯出版社, 2024. 可获取自: <https://www.hanspub.org/journal/paperinformation?paperID=124302>

[3] 赵明. 基于微信小程序的高校校园二手物品交易平台的设计与实现[J]. 科技与创新, 2022(7): 第 16 条 . 可获取自: <https://www.cnki.com.cn/Article/CJFDTotal-KJYX202207016.htm>

[4] 高星宇, 张家骏. 校园二手交易平台的设计与实现[J]. 电子技术与软件工程, 2021(13). 可获取自: <https://dzru.cbpt.cnki.net/EditorIN/WebPublication/paperDigest.aspx?paperID=DZRU202113073>

- [5] 温智宏, 王旖旎, 付妍, 乐凯丽, 陈亮. 基于SpringBoot+Vue的旅游电子商务系统设计与实现 [J]. E-Commerce Letters, 2025. 可获取自 : <https://www.hanspub.org/journal/paperinformation?paperID=126261>
- [6] Fielding R T. Architectural Styles and the Design of Network-Based Software Architectures[D]. University of California, Irvine, 2000: 76–106. 可获取自 : <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [7] You E. Vue.js 3.0 Core Design Principles[EB/OL]. Vue.js Official Documentation, 2020. 可获取自: <https://v3-migration.vuejs.org/>
- [8] Cantelon M, Harter M, Holowaychuk T J, et al. Node.js in Action[M]. 2nd ed. Manning Publications, 2017: 1–30.
- [9] 王珊, 萨师煊. 数据库系统概论（第5版）[M]. 北京: 高等教育出版社, 2014: 1–20, 168–195.
- [10] 张海藩, 牟永敏. 软件工程导论（第6版）[M]. 北京: 清华大学出版社, 2013: 1–15, 108–135.

---

本文系作者毕业设计论文，指导教师：某教授。